



Analisis Kompleksitas Waktu Menggunakan *Sorting Algorithm* pada Pengaplikasian Fitur Pengurutan Harga dari Terendah dan Tertinggi di Shopee

Davina Azalia Tara¹, Ihza Ferdina², M. Stevanza Sylvester³, Muhammad Fiqi Firmansyah⁴,
Muhammad Sulthonul Izza⁵, Nur Widhya Astuti⁶, Rafael Putra Amarta⁷, Ridwan Fajariansyah⁸

¹Teknik Informatika, Universitas Negeri Semarang, vinatara27@students.unnes.ac.id

²Teknik Informatika, Universitas Negeri Semarang, ihzaferdina13@students.unnes.ac.id

³Teknik Informatika, Universitas Negeri Semarang, stevanza08@students.unnes.ac.id

⁴Teknik Informatika, Universitas Negeri Semarang, fiqifirmansyahars@students.unnes.ac.id

⁵Teknik Informatika, Universitas Negeri Semarang, izzaaida213@students.unnes.ac.id

⁶Teknik Informatika, Universitas Negeri Semarang, nwidhya1@students.unnes.ac.id

⁷Teknik Informatika, Universitas Negeri Semarang, rafaelputra@students.unnes.ac.id

⁸Teknik Informatika, Universitas Negeri Semarang, ridwanfajar321@students.unnes.ac.id

Sitasi:

Davina Azalia Tara, D. A., dkk. (2024). Analisis Kompleksitas Waktu Menggunakan *Sorting Algorithm* pada Pengaplikasian Fitur Pengurutan Harga dari Terendah dan Tertinggi di Shopee. *Jurnal Potensial*, Vol. 3, No. 1.

Abstract

Technological developments are increasingly developing, giving rise to innovation by changing the way people shop. Nowadays people can easily search for the items they need. The sorting feature in e-commerce, including Shopee, makes it easier for someone to find the items they need. One can even sort items from lowest price to highest. Each type of algorithm has varying degrees of effectiveness. The effectiveness of an algorithm can be measured by how much time and space (algorithm space/memory) is required to run the algorithm. An effective algorithm can minimize space and time requirements. The less space required to run an algorithm, the more effective the algorithm. This research aims to analyze time complexity using quantitative methods. The result of this article is an analysis of the length of time the sorting algorithm takes to use the Shopee application's price sorting feature from lowest to highest. The benefit provided is knowing how effective the timing of the sorting algorithm is.

Keywords: *sorting algorithm, shopee, lowest price, highest price, effectiveness.*

Abstrak

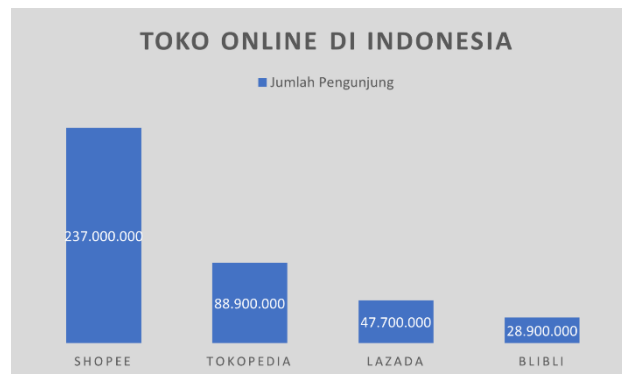
Perkembangan teknologi semakin berkembang sehingga memunculkan sebuah inovasi dengan mengubah cara orang berbelanja. Saat ini orang dapat dengan mudah mencari barang yang diperlukan. Adanya fitur *sorting* di *e-commerce* termasuk Shopee memudahkan seseorang dalam mencari barang yang diperlukan. Bahkan seseorang bisa mengurutkan barang dari harga terendah sampai yang tertinggi. Masing-masing jenis algoritma memiliki berbagai tingkat efektivitas. Efektivitas algoritma dapat diukur dengan berapa banyak waktu dan ruang (*space*/algoritma memori) yang diperlukan untuk menjalankan algoritma. Algoritma yang efektif dapat meminimalkan kebutuhan ruang dan waktu. Semakin sedikit ruang yang dibutuhkan untuk menjalankan sebuah algoritma, yang lebih efektif algoritma. Penelitian ini bertujuan untuk menganalisis kompleksitas waktu menggunakan metode kuantitatif. Hasil dari artikel ini adalah analisis lamanya waktu *sorting algorithm* dalam penggunaan fitur pengurutan harga aplikasi Shopee dari terendah dan tertinggi. Manfaat yang diberikan adalah mengetahui bagaimana keefektifitasan waktu pada *sorting algorithm*.

Kata Kunci: algoritma *sorting*, shopee, harga terendah, harga tertinggi, efektivitas

PENDAHULUAN

Generasi muda tentunya tidak asing dengan berbelanja melalui *online*. Belanja *online* merupakan transaksi jual beli barang atau jasa yang media perantaranya yaitu internet. Banyak keuntungan dari berbelanja *online*, diantaranya tidak perlu mendatangi toko secara langsung, banyak promosi yang ditawarkan, pilihan metode pembayaran yang beragam, dan tidak perlu mengantri. (Rahmadanti, Jajuli, & Purnamasari, 2021). Platform yang menyediakan layanan berbelanja *online* ini dinamakan *e-commerce*. Terdapat beberapa *e-commerce* yang dapat digunakan

diantaranya yaitu Bukalapak, Tokopedia, Lazada, Shopee, Blibli, Tiket.com, Kaskus, OLX, dan lain-lain. Menurut Patimah, E., Ermatita, E., & Chamidah, N. (2021), dengan adanya aplikasi tersebut, membuat manusia lebih sering menghabiskan waktunya untuk berbelanja di *online shop*. Namun pada artikel ini topik yang dibahas yaitu *e-commerce* Shopee.



Berdasarkan data, Shopee menempati peringkat pertama *e-commerce* yang paling sering dikunjungi dengan total 237.000.000 pengunjung (Ahdiat, 2023). Alasan Shopee banyak dikunjungi oleh para konsumen adalah penggunaan fitur Shopee dapat dengan mudah diterima oleh masyarakat, terdapat banyak promosi menarik, serta banyak pilihan harga dan produk yang beragam. Hal ini juga diperkuat oleh sifat penggunaannya Shopee yang termasuk kedalam kelompok *Customer to Customer* (C2C). *Customer to Customer* (C2C) merupakan cara dalam berbisnis dengan konsumen menjual langsung kepada konsumen (Ardiyansyah & Rahayuningsih, 2020). Semakin banyaknya pengguna Shopee, pihak Shopee tentunya harus memperhatikan kepuasan para pengguna dalam menggunakan aplikasi Shopee (Patimah, E., Ermatita, E., & Chamidah, N., 2021).

Ada juga beberapa masalah dalam transaksi penjualan online, terutama bagi pembeli, yang mungkin menemui banyak kendala, seperti kualitas barang yang dijual; barang untuk dibeli. Pembeli melihat gambar tampilan produk yang dijual dan tidak dapat memastikan secara spesifik apakah produk tersebut memenuhi spesifikasi atau kriteria pembelian. Perkiraan waktu penyerahan barang menarik perhatian pembeli terhadap produk atau barang yang akan dibeli; pada saat barang diserahkan kepada pembeli, barang yang diterima tidak sesuai dengan barang yang ditawarkan di website toko online, terlepas dari spesifikasi, jenis atau sifat barang yang dijual; Apabila pembeli mengajukan hak klaim (pengembalian dana) ke website toko online, maka tidak semua penjual toko online bersedia menanggapi barang yang diklaim.

Menurut Philip Kotler (1998:170) mengatakan bahwa, ada lima tahap dalam keputusan pembelian: pengenalan kebutuhan, pencarian informasi, evaluasi *alternative*, keputusan pembelian, paska pembelian (Philip Kotler, 1998:170). Setelah melakukan penilaian maka diambil keputusan membeli atau tidak membeli (Buchari Alma, 2009). Melakukan pencarian informasi merupakan sebuah permasalahan dalam pemanfaatan algoritma di dunia pemrograman yaitu algoritma pengurutan atau *sorting algorithm*, yang sering digunakan dalam proses pengurutan suatu data. Yahya (2014:135) *Sorting* adalah proses menempatkan data ke dalam urutan dan urutan berdasarkan aturan eksklusif setelah sebelumnya telah diurutkan secara acak atau tidak teratur.

Keefektifan dari suatu pemrograman sangat ditentukan oleh pengaplikasian algoritmanya. Algoritma tidak hanya dapat menyelesaikan persoalan dengan benar akan tetapi harus dapat meminimalisir pemakaian waktu dan efisiensinya seperti menurut Nanang Mahrozi & Muhammad Faisal (2023:90) Algoritma yang dianggap efisien adalah algoritma yang dapat mengurangi sebanyak mungkin penggunaan waktu dan ruang. Algoritma seperti itulah yang disebut dengan algoritma yang efisien. Penggunaan sistem komputer elektronik secara efisien untuk mengklasifikasikan data dalam jumlah besar merupakan langkah penting dalam mengurangi jumlah pekerjaan yang diperlukan perangkat elektronik untuk melakukan penelitian ilmiah dan memproses transaksi bisnis (Friend, E. H. 1956: 134-168). Seorang *programmer* seharusnya selalu memiliki beberapa alternatif dalam penggunaan algoritma yang bisa digunakan untuk menyelesaikan suatu permasalahan. Untuk menyelesaikan suatu permasalahan *sorting*

sendiri ada belasan algoritma. Akan tetapi, jika ingin menghasilkan program yang efektif, *programmer* harus memilih algoritma yang mangkus dan sesuai dengan permasalahan yang ingin dipecahkannya.

Dalam melakukan pengurutan harga pada bentuk pengaplikasian aplikasi Shopee. Perlu dilakukan sebuah penyortiran. Menyortir tidak hanya pada pengurutan sebuah harga saja. Namun, beberapa barang berdasarkan nilai, bentuk, atau mungkin warna dari suatu benda yang ingin kita cari bisa melakukan penyortiran pada aplikasi Shopee. Untuk melakukan sebuah penyortiran, ada beberapa langkah langkah yang harus dilakukan. Langkah langkah tersebut dinamakan dengan algoritma. Algoritma adalah rangkaian instruksi terbatas yang terdefinisi dengan baik, biasanya digunakan untuk memecahkan suatu kelas masalah tertentu atau untuk melakukan komputasi (Saptadi & Sari, 2012).

Menurut Miftah Kurnia (2023:33-39) Algoritma pemrograman, sebagai jantung dari teknologi digital, memiliki potensi besar untuk mengubah cara kita menjalani kehidupan sehari-hari. Algoritma adalah serangkaian alur atau langkah-langkah dalam mengatasi suatu masalah dan mengetahui apa masalahnya, apakah masalah itu logis atau sistematis (Nisa, 2022). Algoritma juga bisa diartikan sebagai urutan logis dalam pengambilan keputusan untuk memecahkan masalah (Ilham Fikriansyah, 2023). Algoritma memiliki banyak jenis untuk dipilih. karena masing masing dari algoritma memiliki fungsi dan caranya tersendiri. Algoritma yang efisien adalah algoritma yang meminimumkan kebutuhan waktu dan ruang (Rahayuningsih, P. A., 2016). Algoritma yang terbaik akan kami terapkan pada penerapan aplikasi ini. Pada artikel ini, pembahasan yang dilakukan adalah algoritma pengurutan atau *sorting algorithm* dalam ilmu komputer, algoritma pengurutan adalah algoritma yang menempatkan elemen-elemen daftar ke dalam urutan. Urutan yang paling sering digunakan adalah urutan numerik dan urutan leksikografis, baik menaik atau menurun. Penyortiran yang efisien penting untuk mengoptimalkan efisiensi algoritma lain (seperti algoritma pencarian dan penggabungan) yang memerlukan data masukan dalam daftar yang diurutkan.

Alasan mengapa judul ini dipilih karena pentingnya algoritma dalam perangkat lunak atau pemrograman sehingga perlu untuk dianalisis konsep dari algoritma. Jadi banyak logika pemrograman yang telah dibuat, untuk kasus umum dan juga khusus. Data sequencing dapat digunakan dalam memilah nilai algoritma (pengurutan) yaitu, *selection sort*, *insertion sort*, *shell sort*, dan *bubble sort*. Dari penelitian sebelumnya, terungkap bahwa penggunaan algoritma *Bubble Sort* memiliki keunggulan dalam kesederhanaan maka lebih mudah dipahami, khususnya dalam situasi dengan jumlah data yang relatif kecil, *Selection Sort* dapat menjadi pilihan yang lebih baik daripada *Bubble Sort* (Rizki Saputra, Andryana and Sholihati, 2021). Pada penelitian berikutnya, terdapat temuan bahwa jumlah iterasi pada *Selection Sort* pada setiap pengujian data lebih rendah dibandingkan dengan *Bubble Sort*. Selain itu, waktu eksekusi *Selection Sort* pada setiap pengujian data juga terbukti lebih cepat dibandingkan dengan waktu eksekusi *Bubble Sort* (Roma Rio, Yusman and Febi Eka, 2017). Masing-masing jenis algoritma memiliki berbagai tingkat efektivitas. Efektivitas algoritma dapat diukur dengan berapa banyak waktu dan memori yang diperlukan untuk menjalankan algoritma. Semakin sedikit memori yang dibutuhkan untuk menjalankan sebuah algoritma, sedikit pula waktu yang dibutuhkan untuk menjalankan sebuah algoritma.

Kompleksitas waktu dalam konteks pengurutan dari harga terendah ke tertinggi pada aplikasi Shopee diperlukan karena memberikan pemahaman tentang seberapa efisien algoritma tersebut dalam menyelesaikannya. Dalam pengembangan perangkat lunak, kinerja dan efisiensi sangat penting untuk memastikan apakah aplikasi tersebut berjalan dengan baik. Jika algoritma pengurutan tidak efisien dan memiliki kompleksitas waktu yang tinggi, maka aplikasi Shopee dapat berjalan lambat ketika menghadapi banyaknya harga produk yang perlu diurutkan, ini dapat mengakibatkan penurunan kinerja aplikasi Shopee dan membuat frustrasi bagi pengguna.

Efisiensi dalam pengurutan harga dapat meningkatkan kualitas layanan yang diberikan oleh Shopee kepada pengguna, ketika harga produk diurutkan dengan baik pengguna dapat dengan mudah menemukan harga produk yang sesuai dengan budget yang mereka inginkan, maka dari itu dengan dilakukannya kompleksitas waktu menggunakan *Sorting algorithm* pada fitur harga terendah dan tertinggi pada aplikasi Shopee, kita dapat memprediksi berapa lama *algorithm* akan memakan waktu untuk menyelesaikannya berdasarkan *input* yang diberikan (Putra et al., 2023).

Selain itu, pengguna aplikasi Shopee juga ingin melihat hasil pencarian atau daftar harga produk dengan cepat. Jika pengurutan harga produk memerlukan waktu yang lama, maka pengguna mungkin meninggalkan aplikasi atau beralih ke *platform* lain yang lebih responsif. Dengan memilih algoritma pengurutan harga dengan kompleksitas waktu, memastikan bahwa pengguna dapat dengan mudah menjelajahi produk – produk dan pengalaman berbelanja mereka tidak terganggu oleh lamanya waktu pengurutan.

Penelitian kompleksitas waktu menggunakan metode kuantitatif dimana metode kuantitatif sendiri ialah metode penelitian yang berfokus pada pengumpulan, analisis, dan interpretasi data berupa angka atau variabel yang dapat diukur. Dalam metode ini penelitian sering menggunakan instrumen pengumpulan data seperti kuesioner, survei atau eksperimen untuk mengumpulkan data numerik. Terdapat macam-macam algoritma yaitu *bubble sort*, *insertion sort*, *selection sort*, *shell sort*, *merge sort*, *quick sort*, dan *heap sort*. Pada penelitian ini hanya akan dibahas satu metode pengurutan data yaitu algoritma *bubble sort* untuk membatasi luasnya pembahasan. Hal ini dikarenakan algoritma *bubble sort* adalah algoritma pengurutan paling dasar. Algoritma ini memiliki metode pengurutan paling sederhana daripada algoritma pengurutan yang lain sehingga mudah dipahami terutama bagi yang baru memasuki ranah pemrograman. Selain itu, jumlah pengulangan akan tetap sama jumlahnya meskipun data sudah cukup terurut. Sehingga, penulis memutuskan untuk memilih algoritma *bubble sort* sebagai objek penelitian.

Sorting didefinisikan sebagai pengurutan sejumlah data berdasarkan nilai kunci tertentu. Pengurutan dapat dilakukan dari nilai terkecil ke nilai terbesar (*ascending*) atau sebaliknya (*descending*). Algoritma Sorting termasuk salah satu contoh yang kaya akan solusi (Wisudawan, 2008). Untuk mengatasi masalah kompleksitas waktu yang terkait dengan *sorting algorithm* pada pengaplikasian fitur pengurutan harga dari terendah dan tertinggi di Shopee berikut beberapa solusi yang dapat kami tawarkan berdasarkan hasil penelitian kami (A. Oktian Permana & Sudin Saepudin, 2023).

Mengaplikasikan “MergeSort”

Merge Sort adalah *sorting algorithm* berbasis perbandingan yang banyak digunakan dan dikenal karena kinerjanya yang stabil dan efisien. *Merge sort* adalah salah satu algoritma sorting paling populer yang didasarkan pada prinsip Divide dan Conquer Algorithm (Nisa, 2022). Ini termasuk dalam kategori *algorithm* membagi dan menaklukkan, yang berarti membagi masalah besar menjadi masalah yang lebih kecil, menyelesaikannya, dan kemudian menggabungkan solusi dari masalah kecil tersebut untuk mendapatkan hasil akhir. Dalam kasus *merge sort*, cara kerjanya adalah dengan membagi daftar yang belum diurutkan menjadi dua bagian yang lebih kecil secara rekursif, mengurutkan masing-masing bagian, lalu menggabungkannya kembali menjadi satu daftar yang telah diurutkan.

Mengaplikasikan “Caching”

Mekanisme *caching* adalah strategi atau sistem yang menyimpan data yang sering diakses atau dihitung di lokasi penyimpanan sementara, yang dikenal sebagai *cache*, untuk meningkatkan pengambilan data dan kinerja sistem. Tujuan utama dari *caching* adalah untuk mengurangi kebutuhan untuk mengakses sumber data asli atau melakukan perhitungan yang mahal berulang kali dengan menyediakan cara yang lebih cepat untuk mengakses data yang sering digunakan.

Mengaplikasikan “Lazy Loading”

Lazy Loading adalah teknik yang digunakan dalam pemrograman komputer dan pengembangan web untuk mengoptimalkan pemuatan seperti gambar, skrip, atau data, dengan cara menunda pemuatannya hingga diperlukan. Tujuan utama dari pemuatan lambat adalah untuk meningkatkan kinerja dan pengalaman pengguna dengan mengurangi waktu pemuatan awal dan menghemat bandwidth. *Lazy Loading* umumnya diterapkan pada aplikasi web, namun bisa juga digunakan pada sistem perangkat lunak lain.

Mengaplikasikan “*Optimize Database Queries*”

Optimalkan kueri *database* yang digunakan untuk mengambil data produk. Pengindeksan dan desain *database* yang tepat dapat mengurangi waktu yang dibutuhkan untuk mengambil dan mengurutkan data secara signifikan.

Mengaplikasikan “*Load Balancing*”

Mendistribusikan beban kerja penyortiran ke beberapa server menggunakan teknik *Load Balancing*. Hal ini membantu memastikan bahwa proses penyortiran tidak membebani satu server, sehingga meningkatkan skalabilitas dan waktu respons.

Manfaat *sorting algorithm* bagi masyarakat yaitu dapat memudahkan masyarakat untuk memilah dengan penayaranan harga untuk direkomendasikan produk yang ditawarkan agar pengguna dapat memilih produk sesuai budget yang dimiliki. Pengguna dapat memanfaatkan fitur yang disediakan oleh Shopee untuk mengurutkan harga dari yang terendah sampai yang tertinggi dan sebaliknya.

METODE PENELITIAN

Untuk menganalisis kompleksitas waktu menggunakan *sorting algorithm* pada aplikasi fitur pengurutan harga dari terendah ke tertinggi di Shopee melibatkan beberapa langkah. Ada beberapa algoritma, seperti, algoritma *bubble sort*, algoritma *insertion sort*, dan algoritma *quick sort*. Pada penelitian ini kami menggunakan algoritma *bubble sort* untuk menganalisis kompleksitas waktunya.

Terdapat persamaan dan perbedaan antara algoritma penyortiran kita dengan *parallel merge sort* dan *parallel randomized quicksort*. Algoritma *bubble sort* dan *selection sort* kita menggunakan ide *partition* khusus algoritma yang mirip dengan *quicksort*. Begitu pula, algoritma *insertion sort* kita menggunakan ide *merge* khusus algoritma yang mirip dengan *merge sort*. Berbeda dengan *merge sort* dan *quicksort*, algoritma kita tidak efisien secara komputasi. Hal ini dikarenakan oleh fungsi *partition* dan *merge* yang lambat yang digunakan oleh algoritma kita, yang masing-masing memanggil dirinya sendiri sebanyak empat atau tiga kali. Sebaliknya, fungsi *merge* dari *merge sort* hanya memanggil dirinya sendiri dua kali (Ganapathi, P., & Chowdhury, R. 2021).

Algoritma pengurutan *bubble sort* dilakukan dengan cara membandingkan elemen yang diseleksi dengan elemen berikutnya. Jika elemen yang diseleksi lebih besar dari elemen berikutnya, maka posisi elemen tersebut akan ditukar dengan elemen berikutnya. Setiap langkah dari proses ini seakan-akan menggeser elemen dari kanan ke kiri satu persatu jika barisan disusun secara horizontal. Jika barisan disusun secara vertikal, maka terlihat seperti gelembung-gelembung (*bubble*) yang naik dari dasar akuarium. Itulah mengapa algoritma ini disebut dengan algoritma *bubble sort* (Sitepu, Yusman, & Febriansyah, 2017). Kekuatan dari algoritma bergantung pada nilai-nilai yang berbeda dalam daftar dan di mana ada lebih banyak pengulangan dalam daftar (Obed Appiah & Ezekiel Mensah Martey, 2015).

Di sini kami menggunakan sample data berupa JSON (JavaScript Object Notation) dan mengimplementasikan algoritma *bubble sort* pada bahasa pemrograman python versi ketiga. Dalam data yang berupa JSON, kami menyimpan informasi-informasi, seperti, harga produk, nama produk, merek, dan spesifikasi produk. Kami menggunakan JSON agar mempermudah proses sortir data-data berupa harga produk dan mengurutkannya dari yang terkecil sampai yang terbesar. Dalam artikel ini, kami menggunakan python karena sintaksnya lebih mudah untuk dipahami dan kemudahan mengakses data berupa JSON.

Bubble Sort adalah algoritma pengurutan sederhana yang digunakan dalam komputer untuk mengurutkan sejumlah item data dalam urutan tertentu, biasanya dari yang terkecil hingga yang terbesar (*ascending order*) atau sebaliknya. Algoritma ini termasuk dalam kategori algoritma *swap sort* karena pada setiap iterasi pasangan elemen yang tidak berurutan dipertukarkan hingga seluruh data terurut. Dalam analisa yang dilakukan oleh Rahayuningsih, P. A. (2016), *Bubble sort* algoritma *sorting* paling tua dan sederhana untuk diimplementasikan. Tetapi, algoritma ini juga merupakan algoritma yang paling lamban dan tidak mangkus dibandingkan dengan algoritma pengurutan yang lain dalam penggunaan secara umum.

Pseudocode adalah kode yang ditulis dengan menggunakan sebuah algoritma, tetapi tidak terikat oleh suatu bahasa pemrograman dan tidak terbatas. Pseudocode juga sering diartikan sebagai sintaks yang mendekati bahasa pemrograman tersebut. Bahasa yang digunakan di pseudocode hampir mirip dengan bahasa pemrogramannya. Pseudocode bersifat universal dan lebih ringkas daripada algoritma. Jadi dengan menggunakan pseudocode, kita memiliki gambaran umum dalam mengimplementasikan algoritma bubble sort tersebut.

```

bubbleSort(array)
  for i <- 1 to indexOfLastUnsortedElement-1
    if leftElement > rightElement
      swap leftElement and rightElement
  end bubbleSort

```

Pada contoh *pseudocode* di atas, kami membuat sebuah fungsi bernama *sort_price* yang akan menerima data berupa JSON yang disimpan dalam variabel data bertipe *array*. Setelah itu dilakukan perulangan, untuk melakukan pengecekan satu persatu dari semua data di dalam JSON, karena ada beberapa bagian data, kami hanya akan mengakses data berupa harga. Lalu di dalam perulangan, kami menggunakan kondisi, jika data harga di kiri lebih besar dari data yang berada di kanan, maka diubah urutannya.

HASIL DAN PEMBAHASAN

Asal usul Bubble Sort

Algoritma *bubble sort* kian lama kian populer, karena banyak digunakan di beberapa permasalahan seperti mengurutkan berbagai data untuk mempermudah dalam menentukan kriteria yang akan dipilih. Sehingga waktu yang diperlukan tidak terlalu lama dalam melakukan pencarian data. Menelusuri tentang asal usulnya, algoritma bubble sort dapat ditelusuri kembali di akhir tahun 1950-an. Beberapa sumber mengatakan asal usul dari *bubble sort* ini memiliki banyak pendapat. Ada yang menuliskan bahwa algoritma ini diterbitkan pertama kali diterbitkan oleh Donald Knuth dan ada pula yang menyebutkan bahwa algoritma ini diterbitkan pertama kali oleh Friend. Masing masing pendapat tersebut dapat kita rangkum dengan metode penganalisisan.

Bubble sort ini mungkin berasal dari makalah yang diterbitkan pada tahun 1956 oleh Friend yang berjudul "*Sorting on Electronic Computer Systems*". Dalam makalah tersebut *bubble sort* menggunakan istilah "*sorting by exchange*", yang sangat menggambarkan kategori *Sorting Algorithm*. Pada tahun 1959 sebuah buku muncul yang menerangkan tentang pemrograman. Buku tersebut menjelaskan mengenai gagasan pada pengurutan dengan pertukaran. Yang pada dasarnya buku ini menguraikan konsep dari *bubble sort*. Menariknya buku ini juga membahas tentang penyaranan perbaikan dalam membuat *pass* alternatif ke arah yang berlawanan. Istilah ini serupa digunakan dalam artikel lain tahun 1962. Terdapat salah satu referensi untuk "*bubble sort*" dalam makalah pada tahun 1959, yang membandingkan pengurutan biner dengan "metode pengurutan gelembung yang tidak efisien", meskipun tidak jelas dari mana istilah "*bubble*" muncul. Istilah *bubble sort* tidak benar-benar muncul sampai buku *A Programming Language* karya Iverson tahun 1962 terpublikasi secara luas. Setelah buku tersebut terbit istilah *bubble sort* kemudian diperkuat oleh adanya buku karya Donald Knuth yang terkenal dengan judul '*The Art of Computer Programming : Volume 3 Sorting and Searching (1973)*'. Namun, Knuth tidak memberikan informasi mengenai asal usul bagaimana nama tersebut dibentuk, meskipun ia telah memberikan referensi di tahun 1956 tentang penganalisisan algoritma. Hasil analisis tersebut menghasilkan makalah yang mengacu pada '*sorting by exchange*', tidak membahas lebih lanjut mengenai *bubble sort*. Sebuah bibliografi ekstensif dan rangkaian artikel dari Konferensi ACM tentang penyortiran pada tahun 1962 tidak menggunakan istilah *bubble sort*, padahal algoritma '*sorting by exchange*' telah disebutkan.

Dari banyaknya pendapat yang telah diberikan, dapat disimpulkan bahwa algoritma *bubble sort* memiliki sebuah urutan dari berbagai sumbernya yaitu :

1. Friend, E. H., "Sorting on electronic computer systems", Journal of the ACM, 3(4), pp.134-168 (1956)
2. Iverson, K. E., A Programming Language, John Wiley & Sons (1962)
3. Knuth, D.E., The Art of Computer Programming, Volume 3 Sorting and Searching (1973)

4. Bose, R.C., Nelson, R.J., "A sorting problem", Journal of the ACM, 9(2), pp.282-296 (1962)
5. McCracken, D., Weiss, H., Lee, T. Programming Business Computers. John Wiley (1959)
6. Simiu, E., Filliben, J.J., "Statistical analysis of extreme winds", National Bureau of Standards (NBS), Technical Note 868 (June, 1975)

Klasifikasi Algoritma Pengurutan

Seperti halnya pada pencarian, algoritma pengurutan juga dapat diklasifikan sebagai algoritma pengurutan internal dan algoritma pengurutan eksternal.

1. Algoritma pengurutan internal, yaitu algoritma pengurutan untuk data yang disimpan di dalam memori komputer. Umumnya struktur internal yang dipakai untuk pengurutan internal adalah larik, sehingga pengurutan internal disebut juga pengurutan larik.
2. Algoritma pengurutan eksternal, yaitu metode pengurutan untuk data yang disimpan di dalam *disk storage*, disebut juga pengurutan arsip (*file*), karena struktur eksternal yang dipakai adalah arsip.

Cara Kerja *Bubble Sort*

Cara kerja *Bubble Sort* hampir sama dengan *Selection Sort*, yang membedakan adalah data yang dibandingkan (Sari et al., 2022). *Bubble Sort* adalah salah satu algoritma pengurutan sederhana yang digunakan untuk mengurutkan data-data dalam sebuah *array* atau daftar (Astuti, 2023). Algoritma ini bekerja dengan menempatkan data-data terbesar di dalam lokasi yang tepat di akhir daftar. Proses ini akan diulangi sebanyak yang diperlukan sampai tidak ada lagi pertukaran data-data yang diperlukan, yang mengindikasikan bahwa daftar telah diurutkan (Md. Khairullah, 2013). Dalam mengurutkan data, *bubble sort* bekerja dengan algoritma berikut :

1. Bandingkan $arr[1]$ dengan $arr[2]$ dan susun sehingga $arr[1] < arr[2]$.
2. Bandingkan $arr[2]$ dengan $arr[3]$ dan susun sehingga $arr[2] < arr[3]$.
3. Bandingkan $arr[n-1]$ dengan $arr[n]$ dan susun sehingga $arr[n-1] < arr[n]$ setelah $(n-1)$ kali perbandingan, $arr[n]$ menjadi elemen terbesar pertama terurut.
4. Ulangi step 2 sampai kita telah membandingkan dan memungkinkan untuk menyusun $arr[n-2]$, $arr[n-1]$. Setelah $(n-2)$ melakukan perbandingan, $(n-1)$ akan menjadi elemen terbesar kedua.
5. Dan seterusnya. Langkah ke $(n-1)$
6. Sesudah $(n-1)$ dijalankan, array akan tersusun dalam urutan naik.

Kami melakukan pengujian dengan menggunakan python versi ketiga dengan membuat *variable* yang menyimpan nilai JSON yang sudah kami kumpulkan pada kategori laptop di Shopee serta menambahkan *key* harga, nama, merk, dan spesifikasi. Di dalamnya juga terdapat kode program untuk mengimplementasikan algoritma *bubble sort*, sesuai dengan *pseudocode* yang sudah kami buat.

```
def sort_price(data):
    for i in range(len(data) - 1):
        for j in range(len(data) - 1 - i):
            if data[j]['harga'] > data[j + 1]['harga']:
                temp = data[j]['harga']
                data[j]['harga'] = data[j+1]['harga']
                data[j+1]['harga'] = temp
    return data
```

Kami membuat fungsi *sort_price* yang akan mengambil nilai parameter data, dan menerapkan proses bubble sort dengan mengambil *value* dari *key* harga pada data JSON yang sudah kami buat. Untuk mengukur waktu suatu program berjalan sampai ke proses selesai, kami menggunakan fungsi *time* pada sistem operasi GNU/Linux. Output fungsi *time* terdiri dari tiga bagian, yaitu:

1. *Real* : Waktu yang dihabiskan oleh program untuk berjalan, termasuk waktu yang dihabiskan untuk menunggu proses lain selesai.
2. *User* : Waktu yang dihabiskan oleh program untuk berjalan di mode *user*, yaitu mode di mana program dapat mengakses sumber daya sistem operasi.
3. *Sys* : Waktu yang dihabiskan oleh program untuk berjalan di mode kernel, yaitu mode di mana program dapat mengakses perangkat keras dan sumber daya sistem operasi lainnya.

Kompleksitas Algoritma *Bubble Sort* dapat dilihat dari beberapa jenis kasus, yaitu *worst-case*, *average-case*, dan *best-case*. Kondisi *best-case*, data yang akan disorting telah terurut sebelumnya, sehingga proses perbandingan hanya dilakukan sebanyak $(n-1)$ kali, dengan satu kali *pass*. Proses perbandingan pada *bubble sort* ini hanya dilakukan sebanyak $(n-1)$ kali. Dengan kata lain, pada kondisi *best-case* algoritma *bubble sort* termasuk pada algoritma linear. Sehingga jumlah proses pada kondisi *best-case* dapat dirumuskan sebagai berikut.

$$\text{Jumlah proses} = n - 1$$

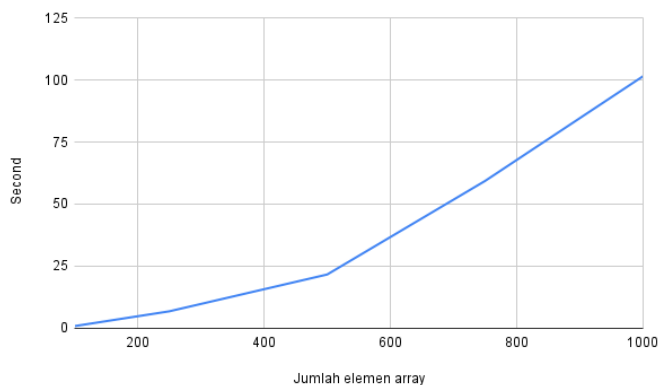
Kondisi *worst-case*, data terkecil berada pada ujung *array*. setiap kali melakukan satu *pass*, data terkecil akan bergeser ke arah awal sebanyak satu step. Dengan kata lain, untuk menggeser data terkecil dari urutan keempat menuju urutan pertama, dibutuhkan *pass* sebanyak tiga kali, ditambah satu kali *pass* untuk memverifikasi. Sehingga jumlah proses pada kondisi *worst-case* dapat dirumuskan sebagai berikut.

$$\text{Jumlah proses} = n^2 + n$$

Pada kondisi *average-case*, jumlah *pass* ditentukan dari data mana yang mengalami pergeseran ke kiri paling banyak. Dengan kata lain, jumlah proses perbandingan dapat dihitung sebagai berikut.

$$\text{Jumlah proses} = x^2 + x$$

Kompleksitas algoritma *Bubble Sort* dapat di gambarkan seperti grafik di bawah ini:



Gambar 1. Grafik kompleksitas algoritma bubble sort

Algoritma *bubble sort* adalah algoritma pengurutan yang tertua dan paling mudah untuk diterapkan. Algoritma ini juga cukup mudah untuk dimengerti. Algoritma ini bekerja dengan cara membandingkan nilai tiap elemen *array* dalam tabel dengan elemen *array* berikutnya, dan menukar nilainya jika sesuai dengan kondisi yang diperlukan terpenuhi. Proses ini akan terus berulang hingga seluruh elemen *array* dalam tabel telah diproses dan elemen *array* dalam tabel telah terurut. pengurutan gelembung ini adalah algoritma yang paling lambat dan paling tidak efisien dibandingkan dengan algoritma pengurutan lain yang umum digunakan. Dalam kasus terbaik (yaitu list sudah terurut), kompleksitas algoritma pengurutan gelembung adalah $O(n)$. Namun, untuk kasus umum, kompleksitas algoritma pengurutan gelembung adalah $O(n^2)$. Dengan interval data antara 100 sampai dengan 1.000 elemen. Waktu eksekusi diukur dengan satuan *Second* (s).

3.4 Kompleksitas Algoritma *Bubble Sort*

Meskipun *bubble sort* dianggap sebagai algoritma yang tidak efisien, algoritma ini memiliki beberapa keunggulan dibandingkan algoritma lain seperti kesederhanaan, kemudahan implementasi dan, kemampuan untuk mengidentifikasi bahwa sebuah daftar sudah diurutkan jika diimplementasikan secara efisien (Obed Appiah & Ezekiel Mensah Martey, 2015). Kompleksitas algoritma *Bubble Sort* dapat dilihat dari beberapa jenis kasus, yaitu *worst-case*, *average-case*, dan *best-case*.

Kondisi *Best case*

Dalam kasus ini, data yang akan disorting telah terurut sebelumnya, sehingga proses perbandingan hanya dilakukan sebanyak $(n-1)$ kali, dengan satu kali *pass*. Proses perbandingan

dilakukan hanya untuk memverifikasi keurutan data. Contoh *Best-Case* dapat dilihat pada pengurutan data "1 2 3 4" di bawah ini.

Pass Pertama

(1 2 3 4) menjadi (1 2 3 4)

(1 2 3 4) menjadi (1 2 3 4)

(1 2 3 4) menjadi (1 2 3 4)

Gambar 2. Kondisi *Best Case*

Dari proses di atas, bisa dilihat bahwa tidak terjadi penukaran posisi sama sekali, sehingga tidak dilakukan *pass* selanjutnya. Perbandingan data dilakukan sebanyak tiga kali. Proses perbandingan pada kondisi ini hanya dilakukan sebanyak $(n-1)$ kali. Persamaan Big-O yang diperoleh dari proses ini adalah $O(n)$. Dengan kata lain, pada kondisi *Best-Case* algoritma *Bubble Sort* termasuk pada algoritma yang sederhana.

Kondisi *Average Case*

Dalam kondisi *average-case*, jumlah *pass* dihitung berdasarkan elemen mana yang mengalami penggeseran ke kiri terbanyak. Proses pengurutan array, misalnya (1 8 6 2), dapat menunjukkan hal ini. Dari (1 8 6 2), jelas bahwa elemen 2 akan mengalami proses penggeseran paling sering, dua kali lipat.

Pass Pertama

(1 8 6 2) menjadi (1 8 6 2)

(1 8 6 2) menjadi (1 6 8 2)

(1 6 8 2) menjadi (1 6 2 8)

Pass Kedua

(1 6 2 8) menjadi (1 6 2 8)

(1 6 2 8) menjadi (1 2 6 8)

(1 2 6 8) menjadi (1 2 6 8)

Pass Ketiga

(1 2 6 8) menjadi (1 2 6 8)

(1 2 6 8) menjadi (1 2 6 8)

(1 2 6 8) menjadi (1 2 6 8)

Gambar 3. Kondisi *Average Case*

Dari proses pengurutan di atas, dapat dilihat bahwa untuk mengurutkan diperlukan dua buah *passing*, ditambah satu buah *passing* untuk memverifikasi. Dengan kata lain, jumlah proses perbandingan dapat dihitung sebagai berikut.

$$\text{Jumlah proses} = x^2 + x$$

Dalam persamaan di atas, x adalah jumlah penggeseran terbanyak. Dalam hal ini, x tidak pernah lebih besar dari n , sehingga x dapat dirumuskan sebagai. Dari persamaan di atas, dapat disimpulkan bahwa notasi big-O nya adalah $O(n^2)$. Dengan kata lain, pada kondisi *average case* algoritma *bubble sort* termasuk dalam algoritma kuadratik.

Kondisi *Worst-Case*

Dalam kasus ini, data terkecil berada pada ujung *array*. Contoh *worst-case* dapat dilihat pada pengurutan data di bawah ini.

Pass Pertama

(4 3 2 1) menjadi (3 4 2 1)
 (3 4 2 1) menjadi (3 2 4 1)
 (3 2 4 1) menjadi (3 2 1 4)

Pass Kedua

(3 2 1 4) menjadi (2 3 1 4)
 (2 3 1 4) menjadi (2 1 3 4)
 (2 1 3 4) menjadi (2 1 3 4)

Pass Ketiga

(2 1 3 4) menjadi (1 2 3 4)
 (1 2 3 4) menjadi (1 2 3 4)
 (1 2 3 4) menjadi (1 2 3 4)

Pass Keempat

(1 2 3 4) menjadi (1 2 3 4)
 (1 2 3 4) menjadi (1 2 3 4)
 (1 2 3 4) menjadi (1 2 3 4)

Gambar 4. Kondisi *Worst Case*

Dari langkah pengurutan di atas, terlihat bahwa setiap kali melakukan satu *pass*, data terkecil akan bergeser ke arah awal sebanyak satu *step*. Kondisi terburuk (*worst case*) tercapai jika data telah urut namun dengan urutan yang terbalik (Saptadi & Sari, 2012). Dengan kata lain, untuk menggeser data terkecil dari urutan keempat menuju urutan pertama dibutuhkan *pass* sebanyak tiga kali, ditambah satu kali *pass* untuk memverifikasi. Sehingga jumlah proses pada kondisi *best-case* dapat dirumuskan sebagai berikut.

$$\text{Jumlah proses} = n^2 + n$$

Dalam persamaan di atas, n adalah jumlah elemen yang akan diurutkan. Sehingga notasi *Big-O* yang didapat adalah $O(n^2)$. Dengan kata lain, pada kondisi *worst-case*, algoritma *bubble sort* termasuk dalam kategori algoritma kuadrat.

Kelebihan dan Kekurangan Algoritma *Bubble Sort*

Setiap algoritma pasti memiliki beberapa kelebihan dan kekurangannya masing-masing. Sama halnya dengan *bubble sort*. *Bubble sort* juga memiliki kelebihan dan kekurangan yang dapat dilihat dari karakteristiknya. Berikut ini beberapa kelebihan dan kekurangan algoritma *bubble sort*.

Kelebihan *Bubble Sort*

Beberapa kelebihan dari algoritma *Bubble Sort* adalah sebagai berikut :

- a. Algoritma yang simpel.

Hal ini terlihat dari proses pengurutan yang hanya menggunakan rekurens dan perbandingan, tanpa menggunakan proses lain. Algoritma pengurutan lain cenderung menggunakan proses lain, misalnya proses partisi pada algoritma *Quick Sort*.

- b. Mudah untuk diubah menjadi kode.

Hal ini diakibatkan oleh simpelnya algoritma *Bubble Sort*, sehingga kecil kemungkinan terjadi kesalahan *syntax* dalam pembuatan kode.

- c. Definisi terurut terdapat dengan jelas dalam algoritma.

Definisi terurut ini adalah tidak adanya satu kalipun *swap* pada satu kali *pass*. Berbeda dengan algoritma lain yang seringkali tidak memiliki definisi terurut yang jelas tertera pada algoritmanya, misalnya *Quick Sort* yang hanya melakukan partisi hingga hanya ada dua buah nilai yang bisa dibandingkan.

- d. Cocok untuk pengurutan data dengan elemen kecil telah terurut.

Cocok untuk pengurutan data dengan elemen kecil telah terurut. Algoritma *Bubble Sort* memiliki kondisi *best case* dengan kompleksitas algoritma $O(n)$.

Kekurangan *Bubble Sort*

Beberapa kekurangan dari algoritma *Bubble Sort* adalah sebagai berikut :

- a. Tidak efektif dalam pengurutan data berskala besar.

Kekurangan terbesar dari algoritma *Bubble Sort* adalah kompleksitas algoritma yang terlalu besar, baik dalam *average case* maupun *worst case*, yaitu $O(n^2)$, sehingga seringkali disebut sebagai algoritma primitif, *brute-force*, maupun algoritma naif.

b. Langkah pengurutan yang terlalu panjang.

Misalnya untuk 1000 buah data, maka akan terjadi proses tidak lebih dari satu juta proses perbandingan. Kompleksitas yang besar ini juga seringkali membuat algoritma *Bubble Sort* sebagai "the general bad algorithm". Bahkan, diantara algoritma pengurutan lain yang memiliki kompleksitas algoritma $O(n^2)$, *insertion sort* cenderung lebih efisien.

PENUTUP

Simpulan

Bubble sort merupakan algoritma pengurutan yang paling simpel dan mudah untuk diaplikasikan di berbagai macam program. Akan tetapi, memiliki tingkat keefisienan waktu yang paling tidak efisien dan sangat lama jika data yang akan diurutkan dalam jumlah besar. Maka, algoritma ini lebih cocok digunakan dalam program yang berskala kecil yang tidak memiliki banyak data yang harus diurutkan. Jika data yang akan diurutkan dalam jumlah banyak maka suatu program akan membutuhkan waktu yang lebih lama untuk memprosesnya. Jika *bubble sort* diaplikasikan pada Shopee maka tidak akan efektif karena data yang ada di Shopee sudah sangat banyak. Solusinya, algoritma yang paling cocok untuk mengurutkan harga pada aplikasi Shopee adalah *quick sort* karena sangat cepat dalam mengurutkan data.

Saran

Terdapat banyak sekali metode yang dapat diaplikasikan untuk menyelesaikan masalah berikut ini yang tentu saja di sesuaikan dengan kebutuhan dan ketersediaan sumber daya. Cepat dan efisien adalah suatu hal yang diharapkan dari algoritma – algoritma tersebut oleh karena itu memilih algoritma dengan ciri tersebut lebih di prioritaskan dibanding dengan algoritma yang lambat dan tidak efisien walaupun lebih sederhana lalu ada juga cara untuk menambah kecepatan penyortiran yaitu dengan variable temporary seperti menurut Achmad Arrosydi (2022:198-203) *Sorting* tercepat didapatkan oleh teknik swapping menggunakan variabel temporary dibandingkan dengan tanpa variabel temporary. Pemilihan algoritma yang tepat akan sangat berpengaruh pada kenyamanan pengguna terutama aplikasi berbasis *E – commerce* yang dimana kecepatan pencarian dan penyortiran berdasarkan keyword yang di input oleh pengguna akan sangat berpengaruh pada kenyamanan pengguna.

Ucapan Terima Kasih

Segala puji dan syukur kami panjatkan kehadirat Tuhan Yang Maha Esa. Karena berkat rahimat dan karunia-Nya, sehingga kami dapat menyelesaikan artikel dengan judul "Analisis Kompleksitas Waktu Menggunakan *Sorting Algorithm* pada Pengaplikasian Fitur Pengurutan Harga dari Terendah dan Tertinggi di Shopee".

Kami sebagai penulis menyadari betul bahwa ada orang-orang yang berjasa dibalik selesainya artikel ini. Tidak ada persembahan terbaik yang dapat kami berikan selain rasa terimakasih kepada pihak yang telah banyak membantu kami. Secara khusus, kami mengucapkan terimakasih kepada Bapak Asep Purwo Yudi Utomo selaku dosen pengampu mata kuliah Bahasa Indonesia yang telah sabar, meluangkan waktu, merelakan tenaga dan pikiran serta turut memberi perhatian dalam memberikan pendampingan selama proses penulisan artikel ini.

Segala kekurangan dan ketidaksempurnaan artikel ini, kami sangat mengharapkan masukan, kritikan, dan saran yang bersifat membangun kearah perbaikan dan penyempurnaan artikel ini. Cukup banyak kesulitan yang kami alami dalam penyusunan artikel ini, tetapi Alhamdulillah dapat terselesaikan dengan baik. Akhir kata, penulis berharap semoga artikel ini dapat bermanfaat bagi semua pihak dan semoga amal baik yang telah diberikan mendapatkan balasan dari Tuhan Yang Maha Esa. Amin.

DAFTAR PUSTAKA

- A. Oktian Permana, & Sudin Saepudin. (2023). Perbandingan algoritma k-nearest neighbor dan naïve bayes pada aplikasi shopee. *Jurnal CoSciTech (Computer Science and Information Technology)*, 4(1), 25–32. <https://doi.org/10.37859/coscitech.v4i1.4474>
- Achmad, A., & Didiet, A. A. (2023). Perbandingan Algoritma Simple Sorting antara Penggunaan Variabel Temporary dan Tanpa Variabel Temporary. *JSisfotek*, 4(4), 198–203. <https://www.jsisfotek.org/index.php/JSisfotek/article/view/185/131>
- Ahdiat, A. (2023, October 11). *Pengunjung Shopee Makin Banyak, Bagaimana E-Commerce Lain?* Retrieved from databoks: <https://databoks.katadata.co.id/datapublish/2023/10/11/pengunjung-shopee-makin-banyak-bagaimana-e-commerce-lain>
- Appiah, O., & Martey, E. (2015). Magnetic Bubble Sort Algorithm. *International Journal of Computer Applications*, 122, 975–8887. <https://doi.org/10.5120/21850-5168>
- Ardiyansyah, A., & Rahayuningsih, P. A. "Penerapan Teknik Sampling untuk Mengatasi Imbalance Class pada Klasifikasi Online Shoppers Intention", Volume 4(1), *Jurnal Teknik Informatika Kaputama (JTIK)*, January 2020, pages 7-14.
- Astuti, Y. (2023). Analisis Pengujian Data Algoritma Bubble Sort. *REMIK: Riset dan E-Jurnal Manajemen Informatika Komputer*, 7(3), 1413–1420. <https://polgan.ac.id/jurnal/index.php/remik/article/view/12594>
- Bose, R. C., & Nelson, R. J. (1962). A Sorting Problem. *Journal of the ACM (JACM)*, 9(2), 282–296. <https://doi.org/10.1145/321119.321126>
- Fikriansyah, I. (2023, July 11). *Algoritma Adalah: Pengertian, Ciri-ciri, dan Berbagai Jenisnya*. DetikEdu. <https://www.detik.com/edu/detikpedia/d-6817970/algoritma-adalah-pengertian-ciri-ciri-dan-berbagai-jenisnya>
- Friend, E. H. (1956). Sorting on Electronic Computer Systems. *Journal of the ACM (JACM)*, 3(3), 134–168. <https://doi.org/10.1145/320831.320833>
- Ganapathi, P., & Chowdhury, R. (2021). Parallel Divide-and-Conquer Algorithms for Bubble Sort, Selection Sort and Insertion Sort. *The Computer Journal*. <https://doi.org/10.1093/comjnl/bxab107>
- Khairullah, M. (2013). Enhancing Worst Sorting Algorithms. *International Journal of Advanced Science and Technology*, 56, 13–26. <https://www.earticle.net/Article/A206946>
- Miftah, K. A. P., & Yahfizam. (2023). Menjelajahi Dunia Algoritma Pemrograman: Membuka Potensi Baru Dalam Kehidupan Sehari-Hari. *Jerkin*, 2(2), 33–39. <https://jerkin.org/index.php/jerkin/article/view/203/127>
- Nanang, F., & Muhammad, F. (2023). ANALISIS PERBANDINAN KECEPATAN ALGORITMA SELECTION SORT DAN BUBBLE SORT. *Scientica* 1(2), 89–98. <https://jurnal.kolibi.org/index.php/scientica/article/view/209/202>
- Nisa, L. B. K. (2022). Penerapan Konsep Kompleksitas Algoritma terhadap Kompleksitas Waktu Algoritma Insertion Sort, Merge Sort, dan Quick Sort. *Makalah IF2120 Matematika Diskrit*. [https://informatika.stei.itb.ac.id/~rinaldi.munir/Matdis/2022-2023/Makalah2022/Makalah-Matdis-2022%20\(132\).pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Matdis/2022-2023/Makalah2022/Makalah-Matdis-2022%20(132).pdf)
- Patimah, E., Ermatita, E., & Chamidah, N. (2021). Analisis Cluster Kepuasan Pengguna Terhadap Layanan Shopee Menggunakan Algoritma K-Means. *Informatik : Jurnal Ilmu Komputer*, 17(3), 209. <https://doi.org/10.52958/iftk.v17i3.3654>
- Putra, J. D., Ananda, M. R., Syahputra, A., Nurshabillah, N., & Sinaga, S. P. (2023). Pengurutan Menggunakan Metode Selection Sort pada Sistem Informasi Lokasi Kontrakan di Kota Medan Berbasis Android. *Blend Sains Jurnal Teknik*, 1(4), 259–266. <https://doi.org/10.56211/blendsains.v1i4.184>
- Rahayuningsih, P. A. (2016). Analisis Perbandingan Kompleksitas Algoritma Pengurutan Nilai (Sorting). Tanggal akses: 23 November 2023 pukul 17.00. <https://www.bing.com/ck/a?!&p=611f88efbe2c5b17jmltdHM9MTcwMDY5NzYwMCZpZ3VpZD0yMmQxOTk1OS1iOTVmlTYwMTetMjc2Zi04YTdkYjgwOTYxZDEmaW5zaWQ9NTE5MA&ptn=3&ver=2&hsh=3&fclid=22d19959-b95f-6011-276f-8a7db80961d1&psq=Analisis+Perbandingan+Kompleksitas+Algoritma+Pengurutan+Nilai>

- [ai+\(Sorting\)&u=a1aHR0cHM6Ly9lam91cm5hbC5ic2kuYWMuaWQvZWp1cm5hbC9pbmRleC5waHAvaXZvbHVzaS9hcnRpY2xiL2Rvd25sb2FkLzcxMi81Nzc&ntb=1](#)
- Rahmadanti, T. F., Jajuli, M., & Purnamasari, I. "Klasifikasi Pengguna Shopee Berdasarkan Promosi Menggunakan Naïve Bayes", Volume 5(2), *Generation Journal*, June 2021 ,Pages 82-83, <https://doi.org/10.29407/gj.v5i2.15998>.
- Rizki Saputra, D.Y., Andryana, S. and Sholihati, I.D. (2021) 'Analisis Perbandingan Algoritma Bubble Sort Dan Selection Sort Pada Sistem Pendukung Keputusan Pemilihan Tempat Kost Berbasis Ios (Iphone Operating System)', *JIPi (Jurnal Ilmiah Penelitian dan Pembelajaran Informatika)*, 6(2), pp. 318-324. Available at: <https://doi.org/10.29100/jipi.v6i2.2015>.
- Roma Rio, S., Yusman, M. and Febi Eka, F. (2017) 'Algoritma Bubble Sort Dan Selection Sort', *Jurnal Komputasi*, 5(1), pp. 81-87. Available at: <https://jurnal.fmipa.unila.ac.id/komputasi/article/viewFile/1484/1228>.
- Saptadi, A. H., & Sari, D. W. (2012). Analisis Algoritma Insertion Sort, Merge Sort Dan Implementasinya Dalam Bahasa Pemrograman C++. *JURNAL INFOTEL - Informatika Telekomunikasi Elektronika*, 4(2), 10. <https://doi.org/10.20895/infotel.v4i2.103>
- Sari, N., Gunawan, W. A., Sari, P. K., Zikri, I., & Syahputra, A. (2022). Analisis Algoritma Bubble Sort Secara Ascending Dan Descending Serta Implementasinya Dengan Menggunakan Bahasa Pemrograman Java. *ADI Bisnis Digital Interdisiplin Jurnal*, 3(1), 16-23. <https://doi.org/10.34306/ABDI.V3I1.625>
- Sitepu, R. R., Yusman, M., & Febriansyah, F. E. (2017). IMPLEMENTASI ALGORITMA BUBBLE SORT DAN SELECTION SORT MENGGUNAKAN ARRAYLIST MULTIDIMENSI PADA PENGURUTAN DATA MULTI PRIORITAS. *Jurnal Komputasi*, 5(1), 81-87. <https://doi.org/10.23960/KOMPUTASI.V5I1.1484>
- Wisudawan, W. F. (2008). Kompleksitas Algoritma Sorting yang Populer Dipakai. <https://informatika.stei.itb.ac.id/~rinaldi.munir/Matdis/2007-2008/Makalah/MakalahIF2153-0708-112.pdf>
- Yahya, Sofyansyah Yusara. 2014. "Analisa Perbandingan Algoritma Bubble Sort dan Selection Sort Dengan Metode Perbandingan Eksponensial." *Jurnal Pelita Informatika Budi Darma*, Vol : VI, No : 3, April 2014.